DATA ENGINEERING & CLOUD · LESSON 5 OF 15

Data Warehouses, Data Marts, Data Lakes, Lakehouses and the Medallion Architecture

Understand the storage choices behind a data platform: warehouse, data mart, data lake and lakehouse. Build a bronze, silver and gold (medallion) layering in code, with a quality gate between layers and a clear rule for what each layer is allowed to contain.

Gap-fill lesson · ✦ 2026 · Intermediate · ⏱ 120 min

By the end of this lesson you can

- Distinguish a data warehouse, data mart, data lake and lakehouse
- Explain the medallion architecture and what each layer holds
- Build bronze, silver and gold layers with a quality gate between them
- Decide which layer a consumer should read from

Datasets used

`superstore_sales.csv`

Four storage patterns

Pattern	What it holds	Strength	Weakness	Typical user
Data warehouse	Structured, modelled tables for reporting	Fast SQL, governance, trust	Costly for raw or unstructured data	Analysts and finance
Data mart	A subject-area slice of the warehouse (sales, HR)	Focused, easy to use	Can duplicate and drift if not governed	One department
Data lake	Raw files of any type on cheap storage	Cheap, flexible, keeps everything	Easy to become a swamp of untrusted files	Engineers and data scientists
Lakehouse	Open table formats on lake storage, with transactions and schemas	One copy serves analytics and machine learning	Requires careful table management	Mixed teams

A **data mart** is usually a set of tables taken from a warehouse for one team. A **data lake** is cheap storage that accepts files without a schema; without discipline it fills with duplicate and undocumented data. A **lakehouse** adds table features (transactions, schema enforcement, time travel) to lake storage, so one copy can serve both dashboards and model training.

CASE STUDY

A lake that nobody trusts

A company dumps every export into one storage bucket. Three years later, nobody knows which of the nine "sales" folders is current, and two dashboards disagree by 4%. The fix is not a bigger bucket; it is layers with rules: raw data kept as it arrived, cleaned data with checked types, and reporting tables that the business has agreed on.

The medallion architecture

The **medallion architecture** organises a lakehouse or warehouse into three layers, each with a stricter contract than the one before:

Layer	Contents	Rules	Who reads it
Bronze	Raw data as it arrived, with load time and source	No changes; append-only	Engineers (for rebuilds)
Silver	Cleaned, typed, deduplicated, conformed data	Types checked; keys unique; quality gate passed	Engineers and analysts
Gold	Business tables and metrics for reporting	Agreed definitions; documented; tested	Business users, dashboards, AI features

Each layer can be rebuilt from the one below it. If a gold table is wrong, you fix the transformation and rebuild gold; you do not patch the table by hand.

Building the layers

The block below builds bronze, silver and gold from the superstore file, with a quality gate between silver and gold. The bronze layer keeps the raw rows unchanged.

Python

```python
import pandas as pd
import duckdb

con = duckdb.connect()
raw = pd.read_csv("superstore_sales.csv", parse_dates=["order_date", "ship_date"])

# Bronze: the raw rows, unchanged, with load metadata
bronze = raw.copy()
bronze["_loaded_at"] = pd.Timestamp("2026-01-15 06:00")
bronze["_source"] = "superstore_sales.csv"
con.register("bronze_src", bronze)
con.execute("CREATE TABLE bronze_sales AS SELECT * FROM bronze_src")

# Silver: typed, deduplicated on the line key, with obvious bad rows removed
con.execute("""
    CREATE TABLE silver_sales AS
    SELECT DISTINCT ON (row_id)
           row_id, order_id, order_date, ship_date, customer_id, product_id,
           CAST(sales AS DOUBLE) AS sales, CAST(profit AS DOUBLE) AS profit,
           CAST(quantity AS INTEGER) AS quantity, region, category
    FROM bronze_sales
    WHERE sales IS NOT NULL AND quantity > 0
    ORDER BY row_id
""")

# Quality gate: the gate must pass before gold is built
gate = con.execute("""
    SELECT
      (SELECT COUNT(*) FROM bronze_sales) AS bronze_rows,
      (SELECT COUNT(*) FROM silver_sales) AS silver_rows,
      (SELECT COUNT(*) FROM silver_sales WHERE row_id IS NULL) AS null_keys,
      (SELECT COUNT(*) - COUNT(DISTINCT row_id) FROM silver_sales) AS duplicate_keys
""").fetchdf()
print(gate)
assert gate["null_keys"][0] == 0 and gate["duplicate_keys"][0] == 0, "quality gate failed"

# Gold: one agreed reporting table, monthly by region and category
con.execute("""
    CREATE TABLE gold_monthly_sales AS
    SELECT DATE_TRUNC('month', order_date) AS month, region, category,
           ROUND(SUM(sales), 2) AS sales, ROUND(SUM(profit), 2) AS profit,
           SUM(quantity) AS units
    FROM silver_sales
    GROUP BY 1, 2, 3
""")
print(con.execute("SELECT * FROM gold_monthly_sales ORDER BY month LIMIT 5").fetchdf())
```

OUTPUT

```
   bronze_rows  silver_rows  null_keys  duplicate_keys
0         6323         6323          0               0
```

```
        month region         category     sales    profit  units
0 2023-01-01  South       Technology  359670.0   8761.61   16.0
1 2023-01-01   East  Office Supplies    2265.0    424.35    9.0
2 2023-01-01  South        Furniture  253729.0  14594.30   24.0
3 2023-01-01   West  Office Supplies   80356.0  10945.12   39.0
4 2023-01-01  North  Office Supplies   16114.5   4138.33   17.0
```

Read the output of the gate: the silver row count can be lower than bronze because bad rows were removed, and the key checks must be zero. A failed assertion stops the pipeline before the gold layer is refreshed, so the reporting table is never built from bad input.

::::warn Do not skip the gate A gold table built directly from bronze is faster to write and fails silently. The gate costs one query per run and catches the errors that would otherwise appear in a board report. ::::

Which layer should a consumer read?

Consumer	Read from	Reason
Executive dashboard	Gold	Agreed definitions and tested
Analyst exploring a new question	Silver	Clean and typed, with detail
Data scientist building a model	Silver (or a feature table derived from it)	Needs detail and history
Engineer rebuilding a table	Bronze	The unchanged source
AI assistant answering business questions	Gold for metrics, silver for detail	Grounded in agreed numbers

Layer rules to write down

1. Bronze: append-only, never edited, with load time and source on every row.
2. Silver: one definition of each entity, types enforced, keys unique, and a documented list of rows removed and why.
3. Gold: each table has an owner, a description, a test on its key and a stated refresh time.
4. A table in one layer is never read by a consumer that should use another layer, unless the exception is written down.

INTERVIEW CORNER

Warehouse or lakehouse: which would you choose?

If the data is structured, the team is SQL-focused, and reporting is the main use, a warehouse is simpler and well governed. If the company also stores files, images or logs, and data scientists and analysts both need the same data, a lakehouse keeps one copy and serves both. Many organisations run a lakehouse for raw and mixed data and a warehouse or gold layer for reporting.

::::key What to remember - Warehouses hold modelled reporting data; lakes hold raw files; lakehouses add table features to lake storage. - Medallion layers: bronze raw and unchanged, silver cleaned and typed, gold agreed and tested. - Put a quality gate between silver and gold, and stop the build when it fails. - Each consumer reads from the layer that matches its trust and detail needs. ::::

Quick quiz

5 questions · tap an answer to check it

1. What does the bronze layer contain?

A Only dashboards

B Raw data as it arrived, unchanged, with load metadata

C Agreed business metrics

D Only the most recent rows

Answer: B. Bronze preserves the source for rebuilds.

2. Which layer should a board dashboard read?

A Bronze

B Any raw file

C Gold

D A personal spreadsheet

Answer: C. Gold holds agreed, tested definitions.

3. Why check for duplicate keys in silver?

A Duplicates make files larger

B Duplicate keys double-count sales in every layer above

C Duplicates are required by the medallion design

D Duplicates are removed automatically

Answer: B. Duplicates inflate totals in gold.

4. What distinguishes a lakehouse from a plain data lake?

A It has no files

B It adds table features such as transactions and schema enforcement to lake storage

C It is always cheaper

D It stores only images

Answer: B. Table formats make lake data reliable enough for analytics.

5. A gold table has a wrong total. What is the correct fix?

A Edit the gold table by hand

B Fix the transformation and rebuild gold from silver

C Delete bronze

D Ignore the difference

Answer: B. Layers rebuild from the layer below, so the fix belongs in the code.

EXERCISE **Add a second quality check**

Extend the silver layer's gate with a check that every silver `order_date` is between the earliest and latest bronze dates, and a check that `sales` is never negative. Make the build fail if either check fails, then deliberately insert one bad row into bronze and confirm the failure.

Show solution (worked answer)

Hints: count rows that break each rule and assert the counts are zero. Insert the bad row in a copy of bronze to avoid changing the raw layer; document the test.

Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.